2/5/2025

ECE 526L Spring 2024 Lab 1 Familiarization with Linux and Synopsys VCS

Muhammad Umar
CSUN ID: 204533244

I hereby attest that this lab report is entirely my own work. I have not copied either code or text from anyone, nor have I allowed or will I allow anyone to copy my work.

Name (printed): **Muhammad Umar**

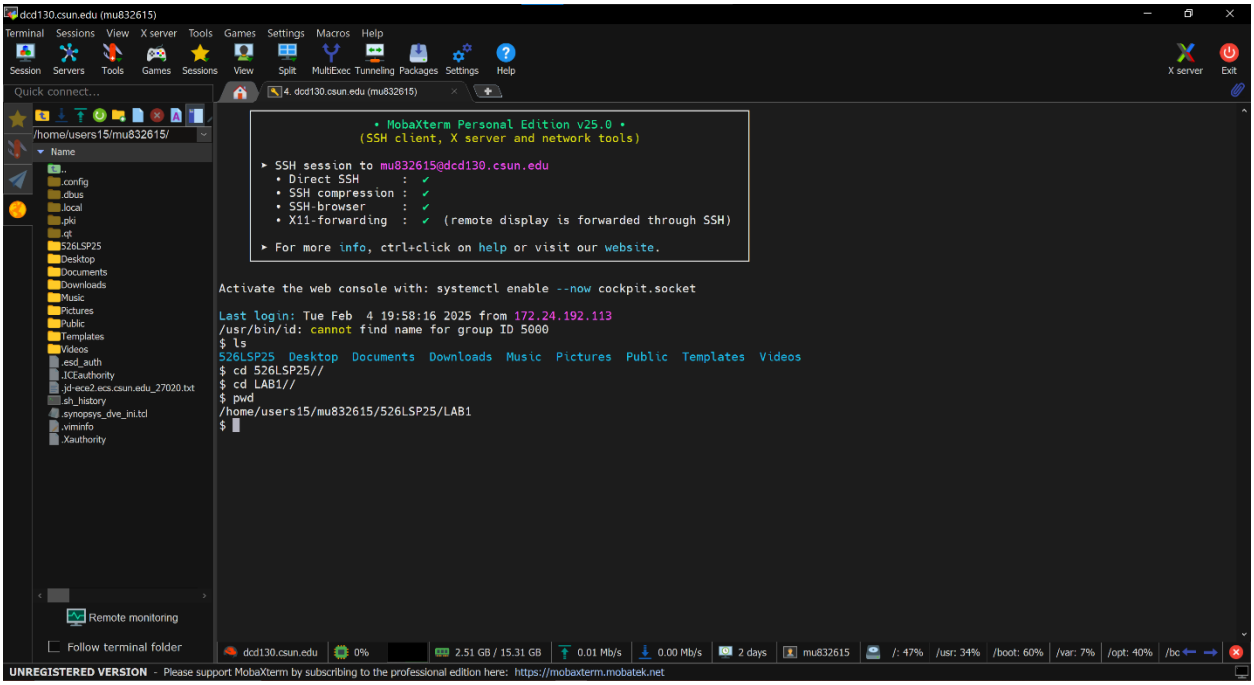
Name (signed):  Date: 02/05/2025

Introduction

This introductory lab objective is to familiarize ourselves with using the Linux operating system and Synopsys VCS to simulate digital designs. Using Verilog, we will simulate a 4x1 multiplexer (MUX) and analyze its functionality using Synopsys VCS. The lab also covers important and most frequently used Linux commands for navigation, file management, and text editing such as **Vim**.

Connecting to the CSUN Server

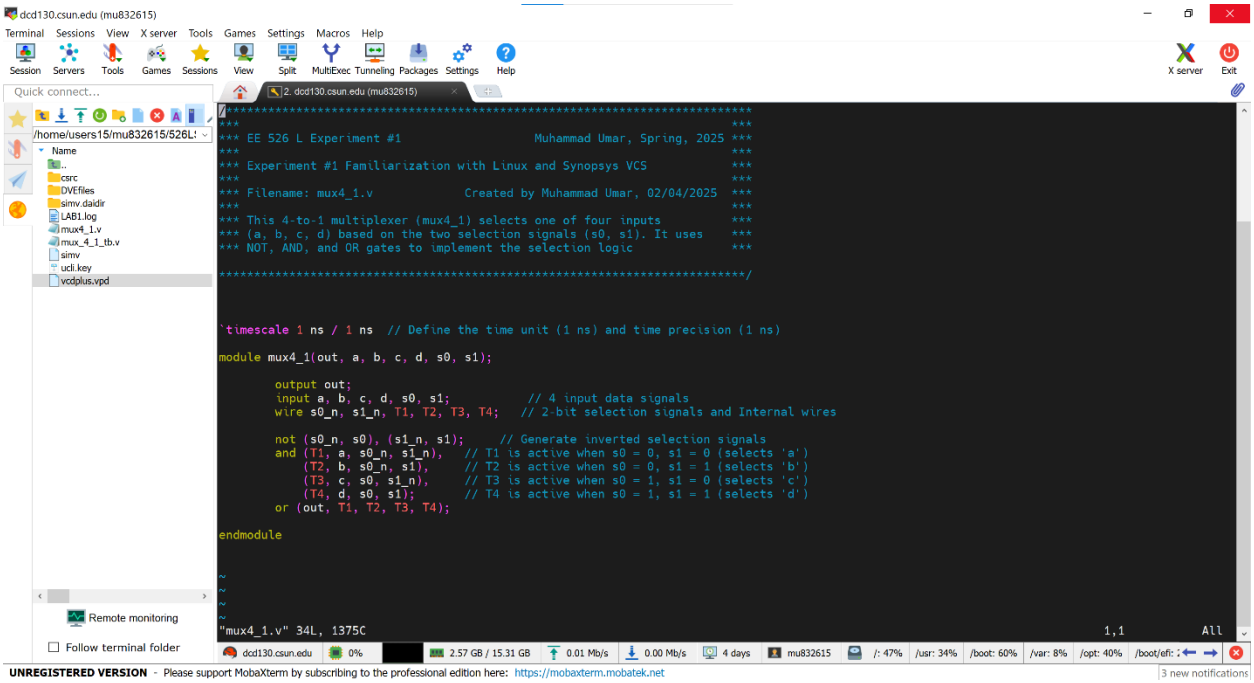
Before starting the lab, we need to connect to a CSUN server using VPN and then **MobaXterm**. The available machines range from 130 -159 and I have selected **dcd130**. After connecting, we navigate to the working directory and create a directory for the lab as mentioned in the below snapshot.



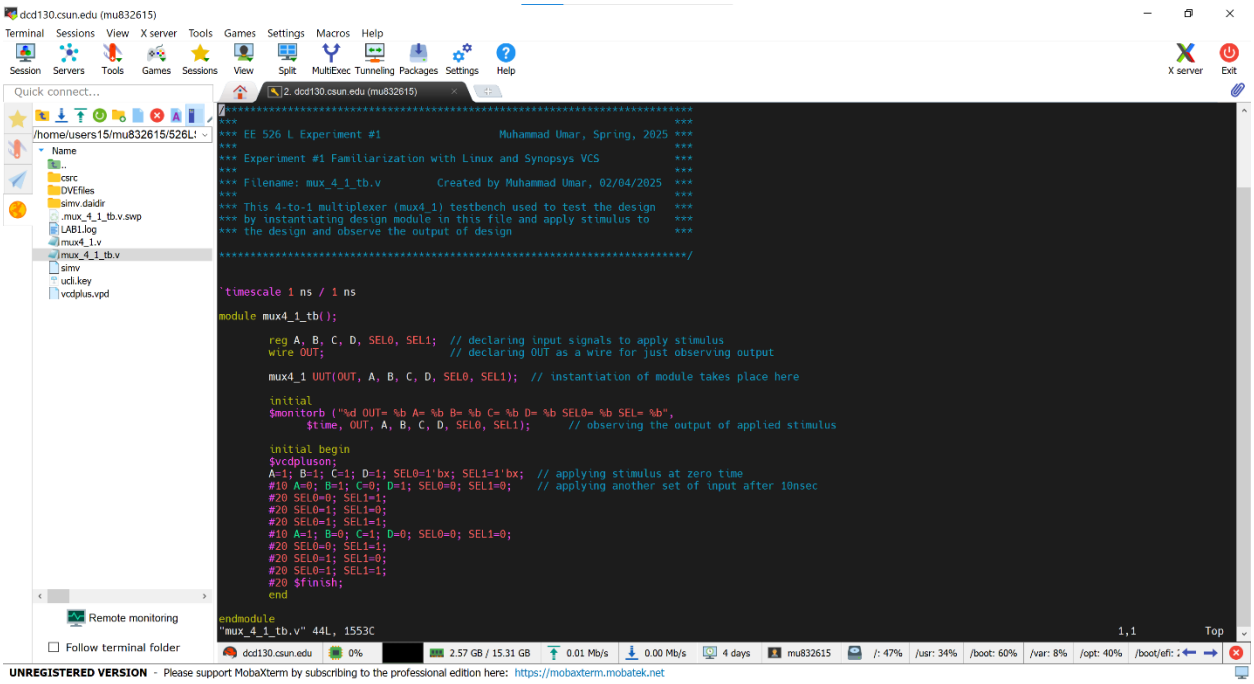
Writing Verilog Code for a 4-input MUX using Vim

I used a text editor called Vim for writing Verilog code and we can access it by using the command **vi**.

MUX Design (mux4_1.v)



MUX Design (mux4_1_tb.v)



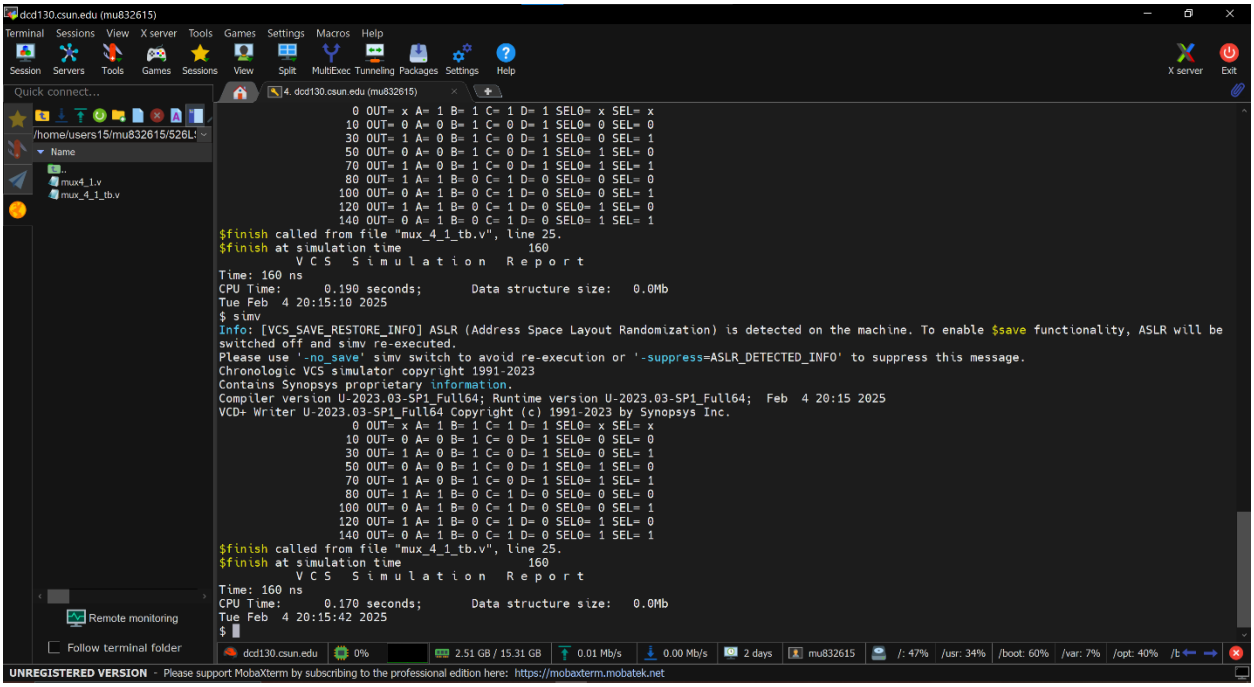
Compiling and Simulating Using VCS

Now compiling the design and testbench file using the following command:

vcs -debug_access+all mux4_1.v mux_4_1_tb.v

Invoking VCS Simulator

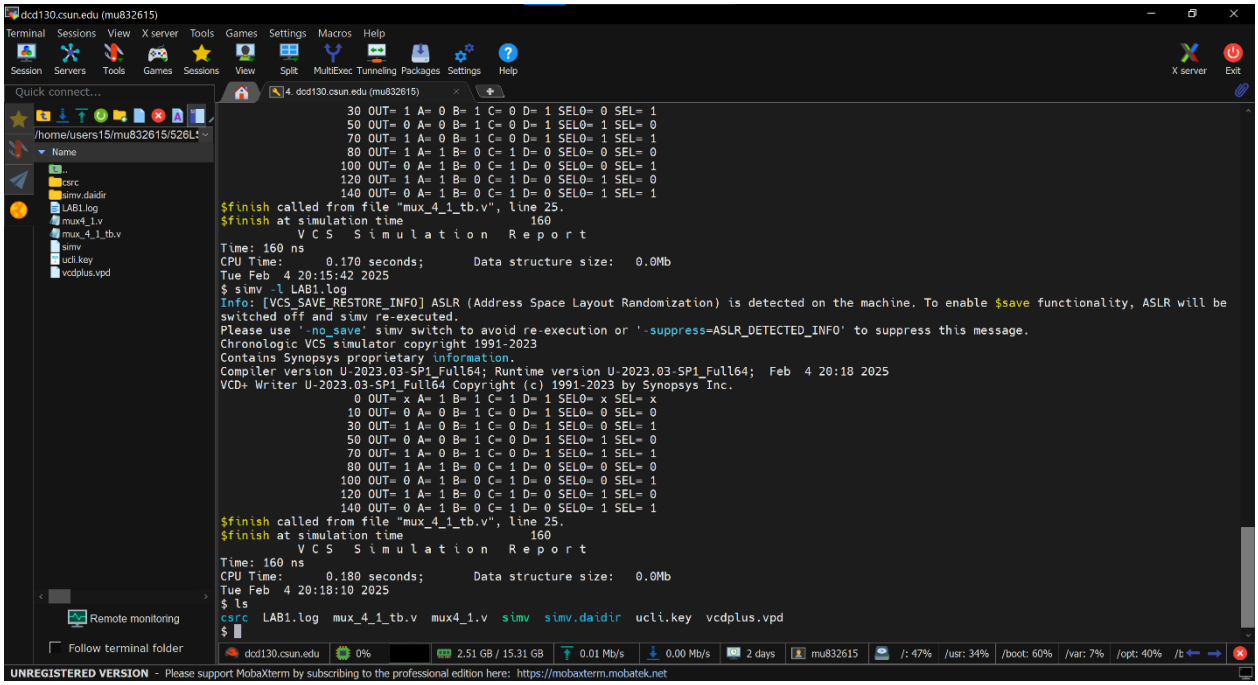
By using **simv** command



Creating Log File

We have to use the following command for log file generation.

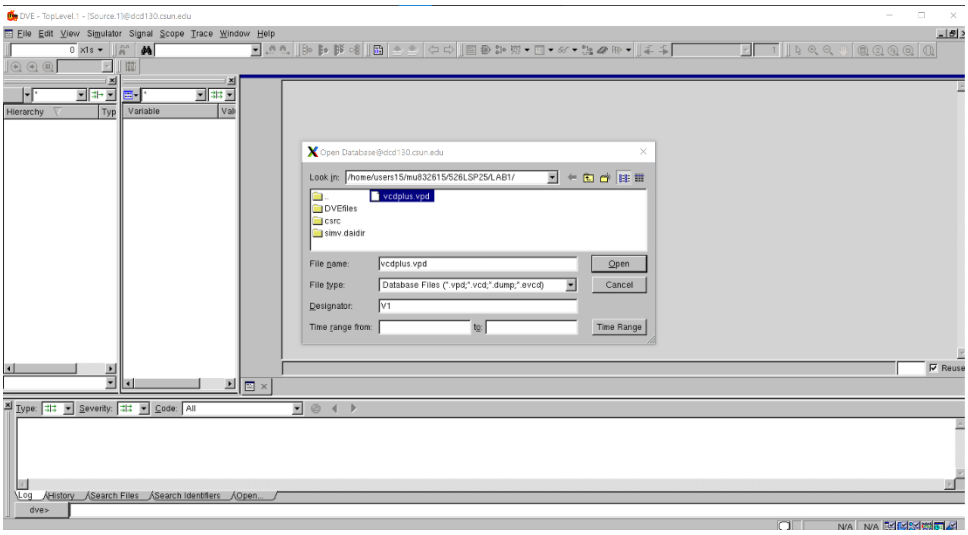
simv –l LAB1.log



Waveform

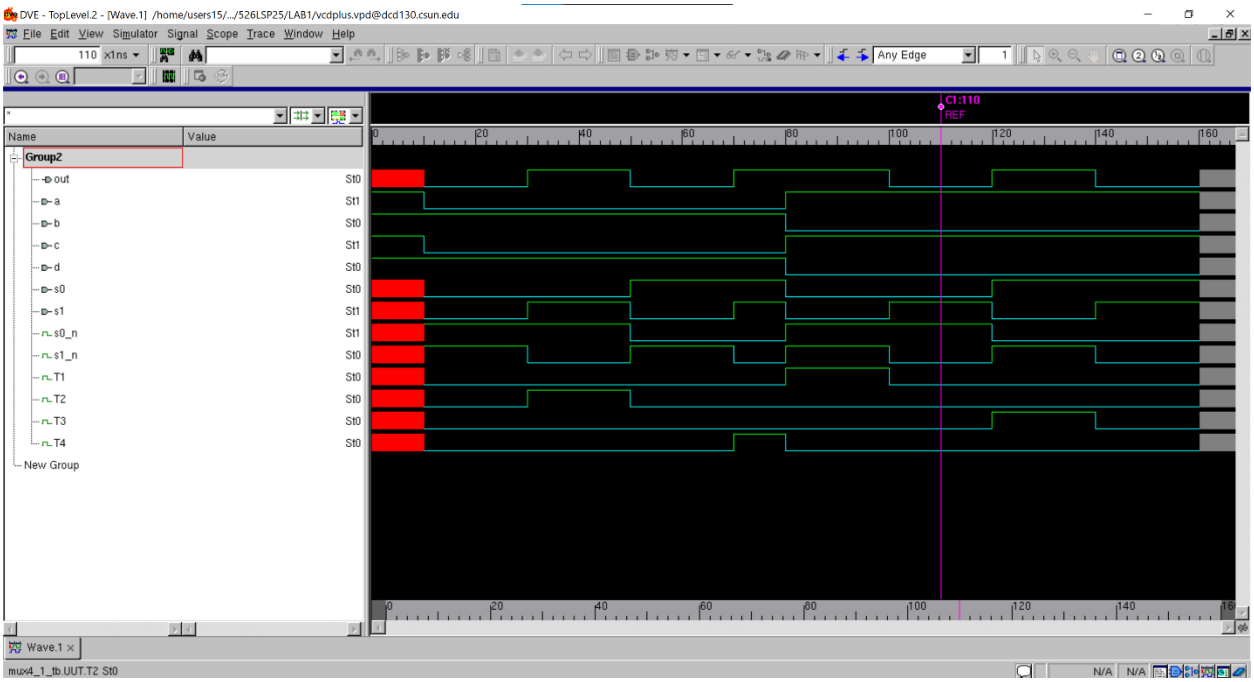
We have to use the following command to display dve and add vpd file for dumping signals in a waveform.

dve -full64



Adding Signals to Waveform

We have to add signals by selecting all signals including the internal signals of the module by getting it from the module after double-clicking it and then adding all signals to the New Wave View.



Source Code

Design File (mux4_1.v)

```
/*****************************************************************************
***                                                                        ***
*** EE 526 L Experiment #1                    Muhammad Umar, Spring, 2025 ***
***                                                                        ***
*** Experiment #1 Familiarization with Linux and Synopsys VCS              ***
***                                                                        ***
*** Filename: mux4_1.v              Created by Muhammad Umar, 02/04/2025  ***
***                                                                        ***
*** This 4-to-1 multiplexer (mux4_1) selects one of four inputs            ***
*** (a, b, c, d) based on the two selection signals (s0, s1). It uses      ***
*** NOT, AND, and OR gates to implement the selection logic                ***
```

```
**************************************************************************/
`timescale 1 ns / 1 ns          // Define the time unit (1 ns) and time precision (1 ns)

module mux4_1(out, a, b, c, d, s0, s1);

        output out;
        input a, b, c, d, s0, s1;           // 4 input data signals
        wire s0_n, s1_n, T1, T2, T3, T4;   // 2-bit selection signals and Internal wires

        not (s0_n, s0), (s1_n, s1);   // Generate inverted selection signals
        and (T1, a, s0_n, s1_n),        // T1 is active when s0 = 0, s1 = 0 (selects 'a')
            (T2, b, s0_n, s1),          // T2 is active when s0 = 0, s1 = 1 (selects 'b')
            (T3, c, s0, s1_n),          // T3 is active when s0 = 1, s1 = 0 (selects 'c')
            (T4, d, s0, s1);            // T4 is active when s0 = 1, s1 = 1 (selects 'd')
        or (out, T1, T2, T3, T4);

endmodule
```

Testbench File(mux_4_1_tb.v)

```
/**************************************************************************
***                                                                     ***
*** EE 526 L Experiment #1                  Muhammad Umar, Spring, 2025  ***
***                                                                     ***
*** Experiment #1 Familiarization with Linux and Synopsys VCS           ***
***                                                                     ***
*** Filename: mux_4_1_tb.v          Created by Muhammad Umar, 02/04/2025 ***
***                                                                     ***
*** This 4-to-1 multiplexer (mux4_1) testbench used to test the design  ***
*** by instantiating design module in this file and apply stimulus to    ***
*** the design and observe the output of design                          ***
```

```
**********************************************************************/


`timescale 1 ns / 1 ns

module mux4_1_tb();

        reg A, B, C, D, SEL0, SEL1;  // declaring input signals to apply stimulus
        wire OUT;                    // declaring OUT as a wire for just observing output

        mux4_1 UUT(OUT, A, B, C, D, SEL0, SEL1);  // instantiation of module takes place here

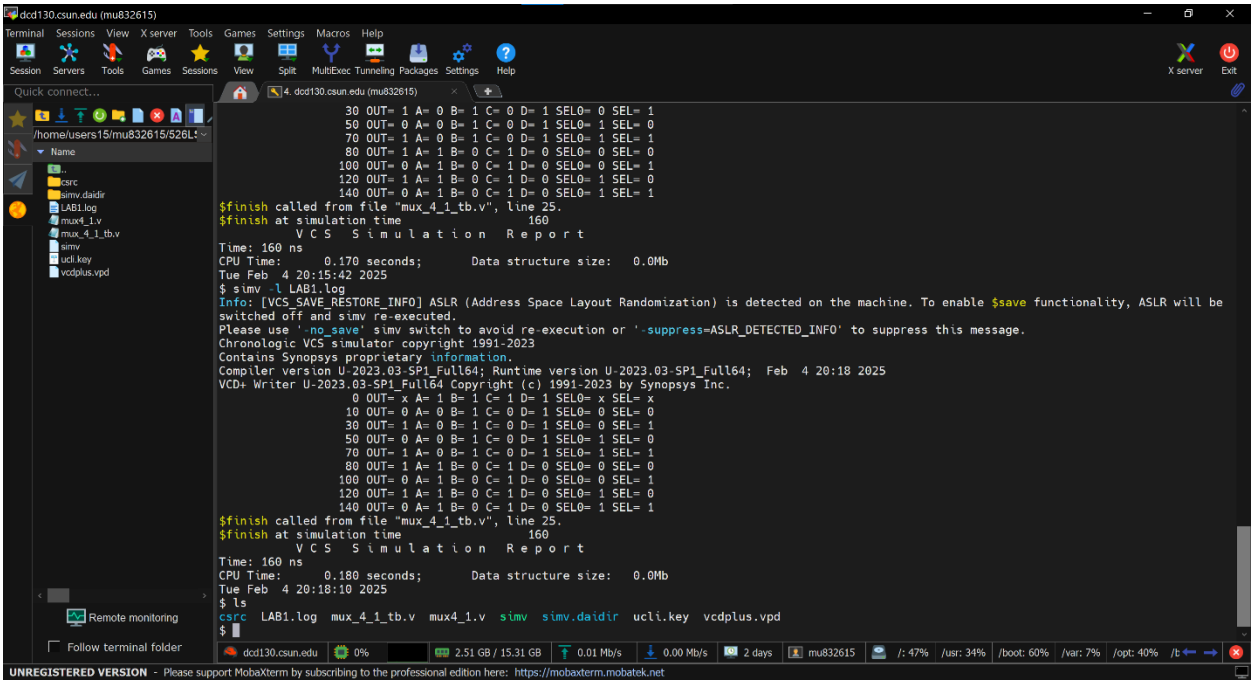
        initial
        $monitorb ("%d OUT= %b A= %b B= %b C= %b D= %b SEL0= %b SEL= %b",
        $time, OUT, A, B, C, D, SEL0, SEL1);         // observing the output of applied stimulus

        initial begin
        $vcdpluson;
        A=1; B=1; C=1; D=1; SEL0=1'bx; SEL1=1'bx;  // applying stimulus at zero time
        #10 A=0; B=1; C=0; D=1; SEL0=0; SEL1=0;              // applying another set of input after 10nsec
        #20 SEL0=0; SEL1=1;
        #20 SEL0=1; SEL1=0;
        #20 SEL0=1; SEL1=1;
        #10 A=1; B=0; C=1; D=0; SEL0=0; SEL1=0;
        #20 SEL0=0; SEL1=1;
        #20 SEL0=1; SEL1=0;
        #20 SEL0=1; SEL1=1;
        #20 $finish;
        end

endmodule
```

RESULT

After simulating the design and verifying it through waveform and log file, we can see from the snapshot that there are no errors noted and results shown below.



Conclusion

We have compiled a Verilog module for a 4:1 multiplexer and checks its functionality using a test bench module by using Synopsys VCS simulator. The simulation results confirmed that the multiplexer operated correctly according to the provided stimulus. This lab provided path that will be followed for the future labs.

Extra Credit Question 1

For a 4-input MUX, there are 4 data inputs, 2 select, and 1 output. We need to consider all possible values for the select lines and data inputs.

- The 2 select lines can take 2^2 = 4 possible combinations.
- The 4 data lines can take 2^4 = 16 possible combinations.
- Each of the 4 data inputs is a single bit, so each can be either 0 or 1.

The total number of test cases is:

Number of test cases = 2^4 × 2^2 = 16×4 = **64.**

64 test cases are needed for an exhaustive test.

Extra Credit Question 2

If we include **X** and **Z** as possible values for the inputs, each input data can now take 4 possible values: 0, 1, X, and Z. The select lines are not affected by **X** and **Z** in the same way as data inputs because they are used to choose which data input reaches the output.

- The 2 select lines can now take 2^2 = 4 combinations.
- Each of the 4 data inputs can take 4 possible values (0, 1, X, Z).

Number of test cases = 4^4 × 2^2 = 256 × 4 = **1024** Number of test cases.

1024 test cases are needed if **X** and **Z** are considered as possible inputs.